



Opening corrupt files

A couple of simple steps are all that are needed for you to open your corrupt files.
<http://dgit.in/UnCorruptFile>



The Best Batman

A debate to put an end to one of the biggest debates playing the DC universe. Which is the best batman? <http://dgit.in/BetBatman>

```
import java.util.*
fun demo(source: List<Int>) {
    val list =
        ArrayList<Int>()
    // 'for'-loops work for
    Java collections:
    for (item in source) {
        list.add(item)
    }
    // Operator conventions
    work as well:
    for (i in 0..source.size -
1) {
        list[i] = source[i] //
get and set are called
    }
}
```

Calling Kotlin from Java

All the functions and properties declared in, let's say, a file named 'example.kt' inside a package 'org.foo.bar', including extension functions, are compiled into static methods of a Java class named 'org.foo.bar.ExampleKt'.

```
// example.kt
package demo
class Foo
fun bar() {
}
// Java
new demo.Foo();
demo.ExampleKt.bar();
```

The name of the generated Java class can be changed using the @JvmName annotation:

```
@file:JvmName("DemoUtils")
package demo
class Foo
fun bar() {
}
// Java
new demo.Foo();
demo.DemoUtils.bar();
```

Having multiple files which have the same generated Java class name (the same package and the same name or the same @JvmName annotation) is normally an error. However, the compiler has the ability to generate a single Java facade class which has the specified name and contains all the declarations from all the files which have that name. To enable the generation of such a facade, use the @JvmMultifileClass

annotation in all of the files that you'll be working with.

5. JAVASCRIPT KOTLIN INTERPORTABILITY

Being a statically typed language, Kotlin still has to interoperate with untyped or loosely typed environments, such as the JavaScript ecosystem. To facilitate these use-cases, the dynamic type is available in the language:

```
val dyn: dynamic = ...
```

The dynamic type basically turns off Kotlin's type checker:

a value of this type can be assigned to any variable or passed anywhere as a parameter;

any value can be assigned to a variable of type dynamic or passed to a function that takes dynamic as a parameter;

null-checks are disabled for such values.

The most peculiar feature of dynamic is that we are allowed to call any property or function with any parameters on a dynamic variable:

```
dyn.whatever(1, "foo", dyn)
// 'whatever' is not defined
anywhere
dyn.whatever(*arrayOf(1, 2, 3))
```

On the JavaScript platform this code will be compiled "as is":
 dyn.whatever(1) in Kotlin becomes
 dyn.whatever(1) in the generated
 JavaScript code.

When calling functions written in Kotlin using values of dynamic type, keep in mind the name mangling performed by Kotlin to JavaScript compiler. You may need to use the @JsName annotation to assign well-defined names to the functions that you need to call.

A dynamic call always returns dynamic as a result, so we can chain such calls freely:

```
dyn.foo().bar.baz()
```

Using Javascript in Kotlin

Kotlin sees Java classes as Kotlin classes, and Java sees Kotlin classes as Java classes. However, JavaScript is a dynamically-typed language,

which means it does not check types in compile-time. You can freely talk to JavaScript from Kotlin via dynamic types, but if you want the full power of Kotlin type system, you can create Kotlin headers for JavaScript libraries.

Inline JavaScript

You can inline some JavaScript code into your Kotlin code using the js("...") function. For example:

```
fun jsTypeOf(o: Any): String {
    return js("typeof o")
}
```

The parameter of js is required to be a string constant just like javascript eval. So, the following code is incorrect:

```
fun jsTypeOf(o: Any): String {
    return js(getTypeOf() + "
o") // error reported here
}
fun get_typeof() = "typeof"
```

Calling Kotlin from Javascript

Kotlin compiler generates normal JavaScript classes, functions and properties you can freely use from JavaScript code. Nevertheless, there are some subtle things you should remember.

Isolating declarations in a separate JavaScript object

To prevent spoiling the global object, Kotlin creates an object that contains all Kotlin declarations from the current module. So if you name your module as myModule, all declarations are available to JavaScript via myModule object. For example:

```
fun foo() = "Hello"
```

Can be called from JavaScript like so:

```
alert(myModule.foo());
```

This is not applicable when you compile your Kotlin module to JavaScript module. In this case there won't be a wrapper object, instead, declarations will be exposed as a JavaScript module of a corresponding kind. For example, in case of CommonJS you should write:

```
alert(require('myModule').
foo());
```

More on Kotlin can be found on the following link: <https://kotlinlang.org/>